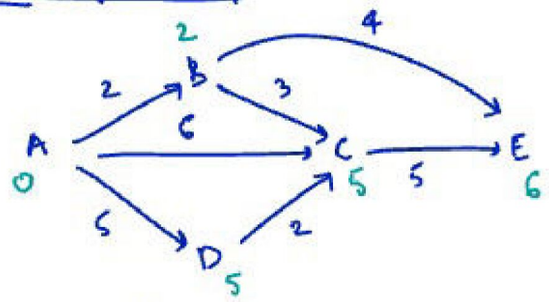
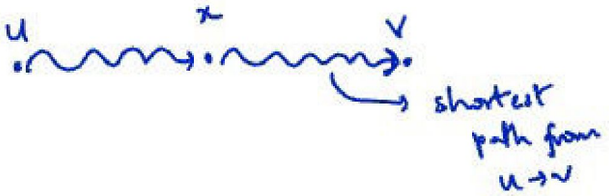
Dynamic Programming



$A \xrightarrow{5} D \xrightarrow{2} C$ length of path = 7

- Shortest paths → optimal substructure



DAG

Directed Acyclic Graph

$f(node)$ → defines the shortest path length from A to that node

$$f(E) = \min\begin{cases} f(B)+4 \\ f(C)+5 \end{cases} = \min\{6, 10\} = 6$$

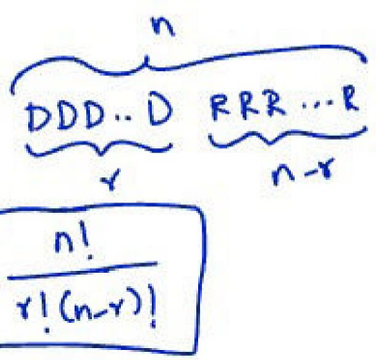
$$f(v) = \min\{ f(u) + c(u,v) \quad \text{for } u \to v \}$$

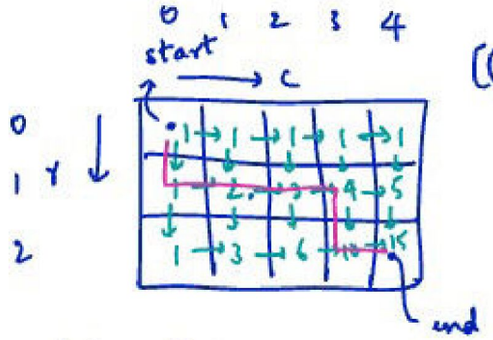
$f(B) = \min\{f(A)+2\} = 2$ $\quad f(s) = 0$

$f(A) = 0$

$$f(C) = \min\begin{cases} f(B)+3 = 5 \\ f(A)+6 = 6 \\ f(D)+2 = 7 \end{cases} = 5$$

$f(D) = \min\{f(A)+5\} = 5$





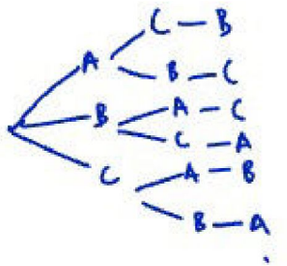
2 Ds & 4 Rs

DRRRDR = $6C_2$

$\frac{6!}{2!4!} = {}^{n}C_r = \frac{n!}{r!(n-r)!}$

[1, 2, 3, 4, 5] ABC = 3!

3 × 2 × 1

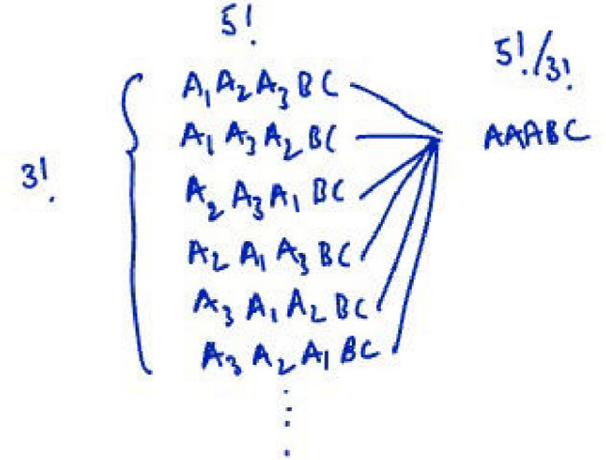


n chars string

n! permutations

AAABC

AAABBC $\frac{6!}{3!2!}$



top down

f(3, 4)

left → f(3, 3)

top → f(2, 4)

Steps

1. f(r, c) := #unique ways to reach (r, c) from the start

   #parameters = 2

   ⇒ DP table is 2D

   dp[r][c] := #unique ways to reach (r, c) from (0, 0)

2. Base case(s): dp[0][0] = 1, dp[r][0] = 1, dp[0][c] = 1

   Recurrence relation (r > 0, c > 0): dp[r][c] = dp[r][c-1] (left) + dp[r-1][c] (top)

3. Order of filling the table: fill row by row from top to bottom, filling each row from left to right

m rows & n cols

4. Final answer: dp[m-1][n-1]

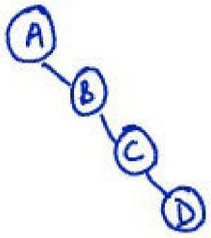
5. R.T.

   #subproblems: mn

   Work per subproblem: $\Theta(1)$

   post processing: $\Theta(1)$

   RT = $\Theta(mn)$



$\approx 2^n$ nodes

DP steps

1. Defining a subproblem
2. Base case & Recurrence relation
3. Order of filling the table
4. How do we compute the final solution from the table.
5. Running time:

   #subproblems = $f(n)$

   # time per subproblem = $g(n)$

   post processing work = $h(n)$

   $RT = f(n) \cdot g(n) + h(n)$

Longest Increasing Subsequence

ABCDE BCD → substring ✓ subsequence ✓
ACD → substring X subsequence ✓

A = [2, -1, 3, 4, 4, 0, 6]
n = #elements

length

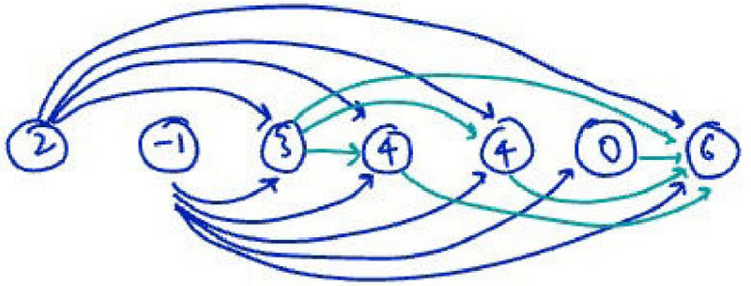
1. Subproblem:
   dp[i] := length of the longest subsequence starting at index 'i'
2. Base case & Recurrence relation
   dp[n-1] = 1
   $dp[i] = 1 + \max\{dp[j] \text{ s.t } i<j\leq n-1 \text{ and } A[i]<A[j]\}$
3. Ordering of filling the table
   from right to left (dp[n-1] to dp[0])
4. Final answer:
   $\max\{dp[i] : 0\leq i<n\}$
5. #subproblems: n
   work per subproblem: O(n)
   post processing: O(n)
   Total time: $O(n^2)$

$\Theta(n^2)$

n-1	1
n-2	1
n-3	2
⋮	
1	n-1
0	n

$\approx \frac{n(n+1)}{2}$

1. Subproblem
   dp[i]: Length of LIS ending at i
2. Base case & recurrence relation
   dp[0] = 1
   $dp[i] = 1 + \max\{dp[j] : 0\leq j<i \text{ \& } A[i]>A[j]\}$
3. Fill from left to right



LIS reduces to longest path in a DAG problem

[0, 1, 2, 3, .. , 9]

[i for i in range(10)]

[i^2 for i in range(1,11)]

[1, 4, 9, ... , 100]

[i x (i+1) for i in range (1, 7, 2)]

[2, 12, 30]

[[0,0,0], [1,1,1]]

num_rows = 2

num_cols = 3

[[i for j in range(nc)] for i in range (num_rows)]

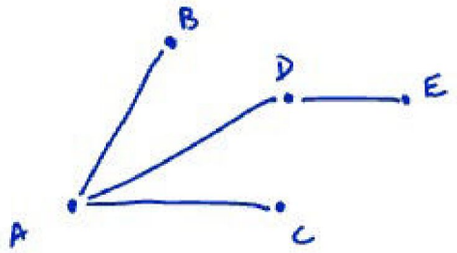
{ 0:"0", 1: "1" , 2:"2", ... , 9: "9" }

{ i: str(i) for i in range(10) }

	0	1	2	3	4	5	6	7
A	2	-1	3	4	4	0	16	4
dp	1	1	2	3	3	2	4	3
prev	-	-	0	2	2	1	3	2

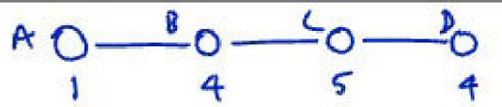
2 3 4 16

Weighted Independent Set



Independent set:
Given an undirected graph $G(V, E)$,
an independent set is a subset of V such that
there are no edges among the vertices in the subset.

Eg:



Path graph:
n vertices
& n-1 edges

Independent sets: $\{B, D, C\}, \{\}, \{A\}, \{B\}, \{C\}, \{D\}, \{E\}$
$\{B, D\}, \{B, C\}, \{B, E\}, \{C, D\}, \{C, E\}$
$\{B, C, E\}, \{A, E\}$

- # independent sets is exponential in terms of 'n' = # vertices

- Max weighted Independent Set
  Given: an undirected $G(V, E)$
  & each vertex has a weight
  Output: Find the independent set with
  max weight.

- Given a solution S,

  case 1: x_{n-1} is not part of $S \Rightarrow W[n-2]$ (due to optimal substructure)

  case 2: x_{n-1} is part of $S \Rightarrow w_{n-1} + W[n-3]$

$W[i] \rightarrow$ Max weight IS using $\{0, \ldots, i\}$ (MWIS)

1. Subproblem

   $dp(i) :=$ weight of MWIS using vertices $\{x_0, x_1, \ldots, x_i\}$

2. Base case

   $dp[0] = w_0$

   $dp[1] = \max\{w_0, w_1\}$

   Recurrence relation:

   $$dp[i] = \max\{dp[i-1], w_i + dp[i-2]\}$$

3. Order of filling:

   from left to right

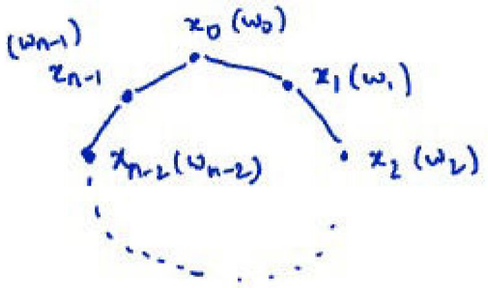
4. Final answer: $dp[n-1]$

5. # subproblems: n

   work per subproblem: $\Theta(1)$

   postprocessing: $\Theta(1)$

   $\Rightarrow$ RT: $\Theta(n)$, space:



Cycle graph

Given an optimal solution S:

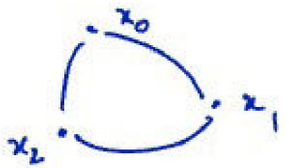
Case 1: doesn't contain x_{n-1} → MWIS in path graph $x_0 - x_1 - \cdots - x_{n-2}$

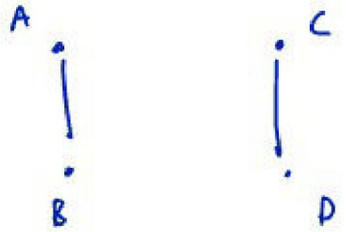
Case 2: does contain x_{n-1} → MWIS in path graph $x_1 - x_2 - x_3 - \cdots - x_{n-4} - x_{n-3}$ + w_{n-1}

dp[i] = max vertices using $\{x_0, x_1, \cdots, x_i\}$ in the original graph

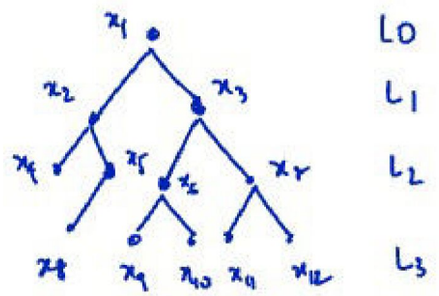
HR1($\{x_0, x_1\}$)

HR1($\{\}$)

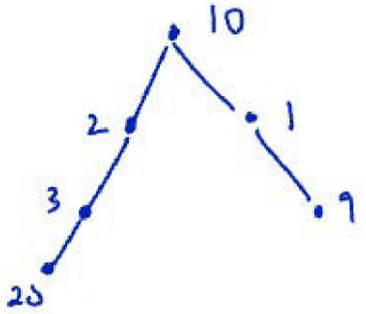




MWIS on trees



optimal



Given an solution S,

Case 1: doesn't contain root $\Rightarrow$ S = MWIS(root.left) + MWIS(root.right)

Case 2: does contain root $\rightarrow$ S = sum{MWIS(root.grandchild)} + root.val

wt of

dp_i(root) := MWIS of subtree including its root

dp_ni(root) := MWIS of subtree excluding its root

dp_i(root) = dp_ni(root.left) + dp_ni(root.right) + root.val

dp_ni(root) = max{dp_i(root.left), dp_ni(root.left)} + max{dp_i(root.right), dp_ni(root.right)}

Final answer = max{dp_i(original_root), dp_ni(original_root)}

dp_i(empty tree) : 0

dp_ni(empty tree) = 0

R.T:

#subproblems: 2n

work per subproblem: $\Theta(1)$ $\Rightarrow$ RT: $\Theta(n)$

post processing : $\Theta(1)$

[2, 3, 1, 4, 9, 6, 2, 8]

Invariants

1. dp[i]: The smallest # that ends a increasing subsequence of length "i+1" among the numbers seen so far.
2. len(dp): Length of the LIS among the number seen so far

[2, 3, 1, 4, 9, 6, 2, 8]

[2 3 4 9 8]
 1 2 6

[2, 4, 4, 6, 6, 6, 8, 8, 10]
 0 1 2 3 4 5 6 7 8

key = 8

solution = 6

1. index of largest # <= key
   index of smallest # >= key
   lower_bound
   upper_bound ...

- Record potential solutions along the way & keep improving the solutions until we run of out of them (we'll make sure to never eliminate better solutions)

} Candidate Variable Pattern

Min coins:

[1, 2, 5, 10] → denominations

$24 \xrightarrow{-2\times10} 4 \xrightarrow{-2\times2} 0$ 4 coins

[1, 8, 10]

$24 \xrightarrow{-2\times10} 4 \xrightarrow{-4\times1} 0$ 6 coins

S: $c_1, c_2, c_3, \ldots, c_k$ min coins = k

$c_k = 10$ $\underbrace{\text{min-coins}(n-10)}_{k-1} + 1$

Target: n

denominations: $d_1, d_2, \ldots, d_m$

dp[i]: min coins required to form 'i' using given denominations

Base case: dp[0] = 0

Recurrence relation: $dp[i] = 1 + \min\{ dp[i-d_j] \ \forall \text{ denomination } d_j \text{ s.t. } d_j \le i \}$

Order of filling the table: left to right

Final answer: dp[n] → size of dp array is 'n+1'

Running time: #subproblems: n+1

work per subproblem: $\Theta(m)$

postprocessing: $\Theta(1)$

$\Rightarrow$ RT: $\Theta(nm) = \Theta(2^p m)$

Pseudo polynomial

n requires 'p' bits to represent

input size = p

$p = \lfloor \lg n \rfloor + 1$ $n \sim 2^p$

8 = 1000

16 = 10000

15 = 1111

$\lg \equiv \log_2$

$\ln \equiv \log_e$

Longest Common Subsequence between 2 strings

AABCADE
AFAAEDC

ABCDF
AECGD

A	B	-	C	-	D	F
A	-	E	C	G	D	-

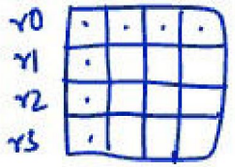
Last column: F or -

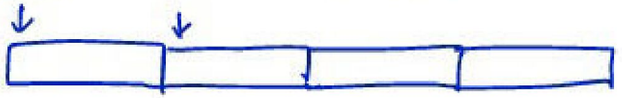
F → LCS(ABCD, AECGD) = 3

\- → LCS(ABCDF, AECG) = 2

2D array

A[m][n]





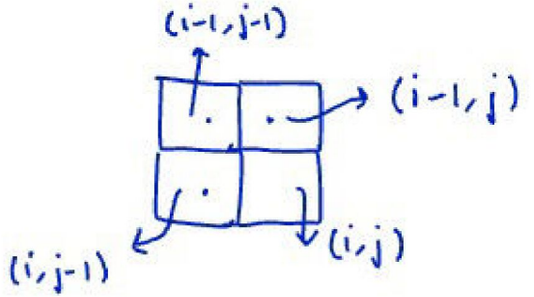
dp(i): MWIS using $x_0, x_1, \dots, x_i$
Order: left to right

dp(i): MWIS using $x_i, x_{i+1}, \dots, x_{n-1}$
order: right to left

Problem: Find length of the LCS between s_1 & s_2
(s_1: m chars, s_2: n chars)

1. dp[i][j]: Length of LCS between $s_1[:i]$ → first i chars of s_1 & $s_2[:j]$ → first j chars of s_2

#rows = m+1 (i goes from 0 to m)
#cols = n+1
(m+1) × (n+1)



2. Base case: dp[i][0] = 0
dp[0][j] = 0

Recurrence:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } s_1[i-1] == s_2[j-1] \\ \max\{dp[i-1][j], dp[i][j-1]\} & \text{otherwise} \end{cases}$$

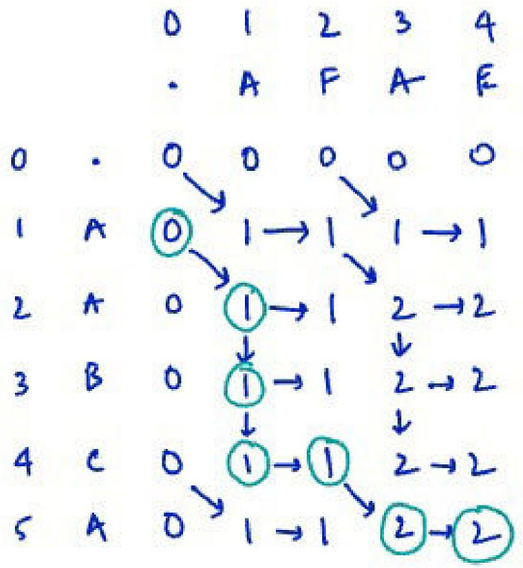
3. Order of filling:
row by row filling each row from left to right

4. Final answer: dp[m][n]

5. RT: #subproblems: (m+1)(n+1) postprocessing: $\Theta(1)$
work per subproblem: $\Theta(1)$ RT: $\Theta(mn)$

S_1 : AABCA $\Rightarrow m = 5$

S_2 : AFAE $\Rightarrow n = 4$



AA